

<i>Nodo</i>
+tick(): Status

NodoSelector
+NodoSelector() +tick(): Status

NodoSecuencia
#hijoEnEjecucion: Nodo
+NodoSecuencia() +tick(): Status

NodoCondicional
#hijo: Nodo #condicion: Supplier<Boolean>
+NodoCondicional(hijo: Nodo, condicion: Supplier<Boolean>) +tick(): Status

NodoCondicion
#condicion: Supplier<Boolean>
+NodoCondicion(condicion: Supplier<Boolean>) +tick(): Status

<i>NodoCompuesto</i>
#hijos: ArrayList<Nodo>
+NodoCompuesto() +addChild(h: Nodo): void

NodoAccion
#accion: Supplier<Integer>
+NodoAccion(accion: Supplier<Integer>) +tick(): Status

Vector2
#coordX: float #coordY: float
+Vector2(x: float, y: float) +getX(): float +getY(): float +setX(x: float): void +setY(y: float): void +sumar(x: float, y: float): void +sumar(v: Vector2): void +multiplicar(e: float): void +normalizar(): void +modulo(): float +direccionHacia(destino: Vector2): Vector2 +clone(): Vector2 +copy(v: Vector2): void +toString(): String

ReproductorSonido
#instancia: ReproductorSonido #musicaFondo: Clip
-ReproductorSonido() +getInstancia(): ReproductorSonido +reproducirSonido(archivo: String): void +reproducirMusicaLoop(archivo: String): void +detenerMusica(): void +pausarMusica(): void +reanudarMusica(): void +setVolumenMusica(volumen: float): void

ConstantesSonidos
-ruta: String = "src" + File.separator + "Assets" + File.separator + "Sonidos" + File.separator +Salto: String = ruta + "Salto.wav" +SonidoBoton: String = ruta + "SonidoBoton.wav" +Aceleracion: String = ruta + "Aceleracion.wav" +FuegoEnemigo: String = ruta + "FuegoEnemigo.wav" +Escalera: String = ruta + "Escalera.wav" +GameOver: String = ruta + "GameOver.wav" +PowerUp: String = ruta + "PowerUp.wav" +NombreConfirmado: String = ruta + "NombreConfirmado.wav" +SumarPuntos: String = ruta + "SumarPuntos.wav" +Disparo: String = ruta + "Disparo.wav" +Victoria: String = ruta + "Victoria.wav" +BolaDeNieveDesaparece: String = ruta + "BolaDeNieveDesaparece.wav" +MuerteJugador: String = ruta + "MuerteJugador.wav" +MusicaFondo: String = ruta + "MusicaFondo.wav"

<i>Observable</i>
+suscribe(suscriptor: Observer): void +unsubscribe(suscriptor: Observer): void +notify(e: Eventos): void +observers(): LinkedList<Observer>

<i>Observer</i>
+update(e: Eventos): void

ObserverColision
#controladorColisiones: ControladorColisiones #elementoObservado: GameElement
+ObserverColision(controlador: ControladorColisiones, elemento: GameElement) +update(e: Eventos): void

ObserverEfecto
#jugador: Jugador #efectoActual: Efectos
+ObserverEfecto(jugador: Jugador) +update(e: Eventos): void -mostrarEfecto(): void -actualizarPosicion(mirada: int): void

ObserverGrafico
#elementoObservado: LogicElement #panelNivel: PanelNivel
+ObserverGrafico(panel: PanelNivel, element: LogicElement) +update(e: Eventos): void #updateSprite(): void #updatePosicionTamaño(): void

ObserverMuerte
#panelNivel: PanelNivel #animal: Enemigo #muerte: int
+ObserverMuerte(enemigo: Enemigo, panel: PanelNivel) +update(e: Eventos): void -mostrarBola(): void

ObserverPuntaje
#entidad: GameElement #puntaje: int
+ObserverPuntaje(enti: GameElement) +update(e: Eventos): void -mostrarPuntaje(): void

HelperLecturaMovimiento
#pantalla: JFrame #jugador: Jugador #keyAdapter: KeyAdapter #direccion: Vector2 #atacar: boolean #salto: boolean #primerTeclaPresionada: boolean #teclasPresionadas: CopyOnWriteArrayList<Tecla>
+HelperLecturaMovimiento(p: JFrame, j: Jugador) #procesarInput(): void +informarJugador(): void +registrarTeclas(): void #cargarTecla(t: Tecla): void #eliminarTecla(t: Tecla): void +reiniciarHelper(): void +primerTeclaPresionada(): boolean

IACalabaza
#DIRECCION_NULA: Vector2 = new Vector2(0, 0) #tiempoCooldownInvocacion: int #cooldownInvocacion: int #rng: Random #sensorVertical: SensorManiobra #visor: VisorJugador #velocidadMovimiento: float #subiendo: boolean #bajando: boolean #alturaObjetivo: float #calabaza: Calabaza
+IACalabaza(calabaza: Calabaza) #inicializarSensores(): void +ejecutar(): void #definirComportamiento(): void #caminoLibre(): boolean #puedeSubir(): boolean #puedeBajar(): boolean #estaSubiendo(): boolean #estaBajando(): boolean #probabilidad50(): boolean #puedeInvocar(): boolean #mover(): int #detener(): int #cambiarDireccion(): int #inicarSubida(): int #subir(): int #inicarBajada(): int #bajar(): int #invocar(): int #aumentarPoder(): int #alturaAlcanzada(dir: int): boolean

IADemonioRojo
#UMBRAL_ATASCO: int = 18 #DIRECCION_NULA: Vector2 = new Vector2(0, 0) #visor: VisorJugador #tiempoUltimaColision: int #atascado: boolean #alturaObjetivo: float #escalando: boolean
+IADemonioRojo(enemigo: Enemigo) +ejecutar(): void #definirComportamiento(): void #inicializarSensores(): void +pisoMismaAltura(): boolean +hayPared(): boolean #puedeEscalar(): boolean #estaAtascado(): boolean #estaEscalando(): boolean #seguirCaminando(): int #cambiarDireccion(): int #movimientoCongelado(): int #detener(): int #caer(): int #iniciarCorreccionMovimiento(): int #corregirMovimiento(): int #iniciarEscalada(): int #escalar(): int #chequearFinEscalada(): boolean #centrarEnemigo(): void #alturaAlcanzada(dir: int): boolean

IAEnemigo
#enemigo: Enemigo #raizComportamiento: NodoCompuesto #direccion: Vector2 #sensor: SensorEntorno
+IAEnemigo(enemigo: Enemigo) #definirComportamiento(): void #inicializarSensores(): void +ejecutar(): void

IAFantasma
#COOLDOWN_CAMBIO_DIRECCION: int = 30 #visor: VisorJugador #timerCambioDireccion: int #velocidadPersecucion: float = 2.5f
+IAFantasma(fantasma: Fantasma) +ejecutar(): void #inicializarSensores(): void #definirComportamiento(): void #puedeCambiarDireccion(): boolean #cambiarDireccion(): int #mover(): int

IAKamakichi
#VELOCIDAD_MOVIMIENTO: float = 2 #VELOCIDAD_MOVIMIENTO_ENFURECIDO: float = 3 #COOLDOWN_ATAQUE: int = 150 #COOLDOWN_ATAQUE_ENFURECIDO: int = 90 #DURACION_PAUSA_POST_ATAQUE: int = 45 #capturadorGolpes: CapturadorGolpes #sensorVertical: SensorManiobra #timerAtaque: int #subiendo: boolean #bajando: boolean #alturaObjetivo: float #ataquesRestantes: int #rng: Random #pausaPostAtaque: int #kamakichi: Kamakichi
+IAKamakichi(jefe: Kamakichi) #definirComportamiento(): void #inicializarSensores(): void +ejecutar(): void #puedelniciarCicloAtaque(): boolean #tieneAtaquesPendientes(): boolean #estaSubiendo(): boolean #estaBajando(): boolean #pasoTiempoPostAtaque(): boolean #estaEnfurecido(): boolean #atacar(): int #prepararAtaques(): int #detenerse(): int #cambiarDireccion(): int #iniciarMovimiento(): int #iniciarSubida(): int #subir(): int #iniciarBajada(): int #bajar(): int #alturaAlcanzada(): boolean

IAMoghera
#COLDOWN_SALTO: int = 120 #UMBRAL_ATASCO: int = 18 #visor: VisorJugadorJefes #sensorVertical: SensorManiobra #tiempoUltimaColision: int #atascado: boolean #alturaObjetivo: float #cooldownVertical: int #moghera: Moghera
+IAMoghera(jefe: Moghera) +ejecutar(): void #definirComportamiento(): void #inicializarSensores(): void #puedelniciarMovimientoVertical(): boolean +movimientoNulo(): int +detectaJugador(): boolean +jugadorDistintaAltura(): boolean +jugadorMasArriba(): boolean +jugadorMasAbajo(): boolean #puedeMoveVertical(): boolean #saltando(): boolean +pisoAbajo(): boolean +pisoArriba(): boolean #cayendo(): boolean +visualizajugador(): boolean +atacar(): int +pisoMismaAltura(): boolean +hayPared(): boolean #movimientoCongelado(): int #detenerse(): int #inicarCaída(): int #caer(): int #iniciarSalto(): int #salto(): int #alturaAlcanzada(dir: int): boolean

IARanaDeFuego
#DIRECCION_NULA: Vector2 = new Vector2(0, 0) #UMBRAL_ATASCO: int = 18 #visor: VisorJugador #tiempoUltimaColision: int #atascado: boolean #alturaObjetivo: float #rana: RanaDeFuego
+IARanaDeFuego(rana: RanaDeFuego) +ejecutar(): void #definirComportamiento(): void #inicializarSensores(): void +visualizajugador(): boolean +atacar(): int +pisoMismaAltura(): boolean +hayPared(): boolean #estaAtascado(): boolean #movimientoCongelado(): int #detener(): int #seguirCaminando(): int #cambiarDireccion(): int #caer(): int #detenerse(): int #iniciarCorreccionMovimiento(): int #corregirMovimiento(): int #chequearFinEscalada(): boolean #alturaAlcanzada(dir: int): boolean

IATrollAmarillo
#COLDOWN_SALTO: int = 60 #UMBRAL_ATASCO: int = 18 #DIRECCION_NULA: Vector2 = new Vector2(0, 0) #visor: VisorJugador #sensorVeritcal: SensorManiobra #alturaObjetivo: float #distanciaObjetivo: float #cooldownSalto: int #tiempoUltimaColision: int #atascado: boolean #troll: TrollAmarillo
+IATrollAmarillo(troll: TrollAmarillo) +ejecutar(): void #definirComportamiento(): void #inicializarSensores(): void +detectaJugador(): boolean +jugadorDistintaAltura(): boolean +jugadorMasArriba(): boolean +jugadorMasAbajo(): boolean #puedeMoveVertical(): boolean +pisoMismaAltura(): boolean +pisoAbajo(): boolean +pisoArriba(): boolean #estaAtascado(): boolean +hayPared(): boolean +caminoLibre(): boolean #saltando(): boolean #cayendo(): boolean #seguirCaminando(): int #cambiarDireccion(): int #iniciarCorreccionMovimiento(): int #corregirMovimiento(): int #movimientoCongelado(): int #detener(): int #mirarJugador(): int #inicarCaída(): int #caer(): int #iniciarSalto(): int #salto(): int #alturaAlcanzada(dir: int): boolean

Estado
#estrategia: EstrategiaMovimiento #entidad: NoEstatico
+Estado(e: NoEstatico) +getEstrategiaMovimiento(): EstrategiaMovimiento

Escalando
+Escalando(e: NoEstatico)

Normal
+Normal(e: NoEstatico)

Realentizado
+Realentizado(e: NoEstatico)

SuperVelocidad
+SuperVelocidad(e: NoEstatico)

EstrategiaMovimiento
#entidad: NoEstatico #velocidadCaminar: float #velocidadSalto: float #GRAVEDAD: float = 0.5f
+EstrategiaMovimiento(e: NoEstatico) +actualizarMovimiento(dir: Vector2, salto: boolean): void +efectuarMovimiento(): void +getVelocidadCaminar(): float +getVelocidadSalto(): float

EstrategiaEscalando
#velocidadEscarar: float = -3f #velocidadBajar: float = 2f #velocidadCaminar: float = 3f
+EstrategiaEscalando(e: NoEstatico) +actualizarMovimiento(dir: Vector2, salto: boolean): void +efectuarMovimiento(): void

EstrategiaNormal
+EstrategiaNormal(e: NoEstatico)

EstrategiaRealentizado
+EstrategiaRealentizado(e: NoEstatico)

EstrategiaSuperVelocidad
+EstrategiaSuperVelocidad(e: NoEstatico)

Juego
#modo: ModoDeJuego #numeralModo: int #fabricaGE: FabricaGameElement #controlGrafica: ControladorGraficas #controladorColisiones: ControladorColisiones #hiloJuego: HiloJuego #puntaje: int #rankingClasico: Ranking #rankingContrarreloj: Ranking #rankingSupervivencia: Ranking #sonido: ReproductorSonido = ReproductorSonido.getInstance() #esFabricaClasica: boolean #nombreTemporal: String
+Juego(cg: ControladorGraficas) +accionarDominioClasico(): void +accionarDominioPersonalizado(): void +getRankingPorModo(modo: int): Ranking +iniciar(): void +setNombreJugador(nombre: String): void +getNombreJugador(): String +reiniciarHilo(): void +finalizar(): void +ganaste(): void #detenerHilo(): void +configurarModo(m: int): void +reiniciarModo(): void +getModoDeJuego(): ModoDeJuego +getControladorGrafica(): ControladorGraficas +getControladorColisiones(): ControladorColisiones +esClasica(): boolean +getFabricaGameElements(): FabricaGameElement +getJugador(): Jugador +getPuntaje(): int +agregarJugadorAlRanking(): void +guardarRankings(): void +cargarRankings(): void +getRankingClasico(): Ranking +getRankingContraReloj(): Ranking +getRankingSupervivencia(): Ranking

ControladorColisiones
#listaColisionables: List<GameElement> #lock: ReadWriteLock = new ReentrantReadWriteLock()
+ControladorColisiones() +buscarColisiones(elemento: GameElement): void +buscarColisiones(sensor: Sensor): void #resolverColision(colisionador: Colisionador, colisionable: Colisionable): void +cargarElementoColisionable(elemento: GameElement): void +eliminarElementoColisionable(elemento: GameElement): void +limpiarColisionables(): void

ElementsRegister
#controlGrafica: ControladorGraficas #controladorColisiones: ControladorColisiones
+ElementsRegister(cg: ControladorGraficas, cc: ControladorColisiones) +registrarObserverGrafico(ge: GameElement): void +registrarObserverColision(ge: GameElement): void +registrarObserverEfecto(j: Jugador): void +registrarObserverMuerte(e: Enemigo): void +registrarObserverPuntaje(g: GameElement): void +registrarElementoColisionable(ge: GameElement): void +registrarElemento(e: GameElement): void +registrar(e: Estatica): void +registrar(e: Enemigo): void +registrar(e: Proyectoil): void +registrar(e: Jugador): void +registrar(p: PlataformaMovil): void

ModoDeJuego
#juego: Juego #creadorNiveles: CreadorDeNiveles #rutasNiveles: LinkedList<String> #nivelActual: Nivel #nivelActualIndex: int #tiempoPartida: int
+ModoDeJuego(creador: CreadorDeNiveles) +iniciarNivel(): void #cargarNivel(indice: int): void +getNivel(): Nivel +getJuego(): Juego +avanzarNivel(): void +actualizarUI(): void +tickTimer(): void +permiteCalabaza(): boolean

ModoClasico
+ModoClasico(creador: CreadorDeNiveles) +avanzarNivel(): void +actualizarUI(): void +tickTimer(): void +permiteCalabaza(): boolean

ModoContrarreloj
#TIEMPO LIMITE: int = 300 +ModoContrarreloj(creador: CreadorDeNiveles) +avanzarNivel(): void +actualizarUI(): void +tickTimer(): void +permiteCalabaza(): boolean

ModoSupervivencia
#PUNTAJE OBJETIVO: int = 10000 #TIEMPO SPAWN: int = 20 #oleadaActual: int = 1 #spawnTimer: int
+ModoSupervivencia(creador: CreadorDeNiveles) +avanzarNivel(): void +actualizarUI(): void +tickTimer(): void +permiteCalabaza(): boolean

Nivel
#lock: ReadWriteLock = new ReentrantReadWriteLock() #CANTIDAD_MAX_ELEMENTOS: int = 208 #FILAS: int = 13 #COLUMNAS: int = 16 #listaEstaticos: CopyOnWriteArrayList<Estatica> #listaEnemigos: CopyOnWriteArrayList<Enemigo> #listaProyectiles: CopyOnWriteArrayList<Proyectoil> #jugador: Jugador #enemigosRestantes: int #modo: ModoDeJuego #tiempoTranscurrido: int #calabazaAparecida: boolean #tiempoAparicionCalabaza: int
+Nivel() +setModo(modo: ModoDeJuego): void +getFilas(): int +getColumnas(): int +obtenerEstaticos(): CopyOnWriteArrayList<Estatica> +obtenerEnemigos(): CopyOnWriteArrayList<Enemigo> +obtenerProyectiles(): CopyOnWriteArrayList<Proyectoil> +eliminarEstatico(elemento: Estatica): void +eliminarEnemigo(enemigo: Enemigo): void +eliminarProyectoil(proyectoil: Proyectoil): void +setJugador(jugador: Jugador): void +getJugador(): Jugador +setEnemigosRestantes(enemigos: int): void +agregarEnemigosRestantes(enemigos: int): void +getEnemigosRestantes(): int +agregarElemento(e: GameElement): void +registrar(e: Estatica): void +registrar(e: Enemigo): void +registrar(e: Proyectoil): void +registrar(e: Jugador): void +registrar(p: PlataformaMovil): void +reiniciarNivel(): void -aparecerCalabaza(): void +tickNivel(): void

Ranking
#topJugadores: ArrayList<RegistroJugador>
+Ranking() +agregarJugador(j: RegistroJugador): void +getTopJugadores(): ArrayList<RegistroJugador>

RegistroJugador
#nombre: String #puntaje: int
+RegistroJugador(nombre: String, puntaje: int) +getNombre(): String +getPuntaje(): int +compareTo(otroJugador: RegistroJugador): int

ControladorGraficas
+registraControladorJuego(controladorJuego: ControladorJuego): void +registrarGameElement(entidad: LogicElement): Observer +mostrarPantallaMenu(): void +mostrarNivel(): void +gameOver(): void +getPanelNivel(): PanelNivel +limpiarGraficaNivelActual(): void +reiniciarHelperDelInput(): void +getHelper(): HelperLecturaMovimiento +victoria(): void

ControladorJuego
+iniciar(): void +finalizar(): void +getJugador(): Jugador +reiniciarModo(): void +configurarModo(m: int): void +getModoDeJuego(): ModoDeJuego +agregarJugadorAlRanking(): void +getRankingClasico(): Ranking +getRankingContraReloj(): Ranking +getRankingSupervivencia(): Ranking +guardarRankings(): void +accionarDominioPersonalizado(): void +accionarDominioClasico(): void +setNombreJugador(s: String): void

ControladorVistas
+accionarInicioJuego(): void +accionarPanelPuntajes(): void +accionarPanelModoDeJuego(): void +accionarPanelMenu(): void +cambiarModoDeJuego(modos: int): void +getControladorJuego(): ControladorJuego +setNombreJugadorTemporal(nombre: String): void

Colisionable
+aceptarColision(c: Colisionador): void

Colisionador
+colisionar(bolaF: BolaDeFuego): void +colisionar(bomba: Bomba): void +colisionar(bolaN: BolaDeNieve): void +colisionar(fuego: Fuego): void +colisionar(P: PowerUp): void +colisionar(e: Enemigo): void +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +colisionar(e: Escalera): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void

Lanzador
+lanzar(direccion: Vector2): void

LogicElement
+getPosicion(): Vector2 +getSprite(): Sprite +getWidth(): int +getHeight(): int +getZOrder(): int

Movable
+mover(): void

Registrable
+aceptarRegistro(r: Registrador): void

Registrador
+registrar(e: Estatica): void +registrar(e: Enemigo): void +registrar(e: Proyectil): void +registrar(e: Jugador): void +registrar(p: PlataformaMovil): void

GUI
#panelMenu: PanelMenu #panelModos: PanelModos #panelRanking: PanelRankings #panelNivel: PanelNivel #panelNombre: PanelNombre #panelGameOver: PanelGameOver #panelVictoria: PanelVictoria #pantalla: JFrame #controladorJuego: ControladorJuego #helper: HelperLecturaMovimiento #nombreJugadorTemporal: String #sonido: ReproductorSonido = ReproductorSonido.getInstance()
+GUI() -registrarFuentePersonalizada(): void +inicializarHelper(): void +reiniciarHelperDelnput(): void #configurarPantalla(): void +registraControladorJuego(cj: ControladorJuego): void +mostrarPantallaMenu(): void +mostrarNivel(): void +mostrarPantallaRanking(): void +mostrarPanelModos(): void +registrarGameElement(entidad: LogicElement): Observer +setNombreJugadorTemporal(nombre: String): void +accionarInicioJuego(): void +accionarPanelPuntajes(): void +accionarPanelModoDeJuego(): void +cambiarModoDeJuego(modo: int): void +accionarPanelMenu(): void +getControladorJuego(): ControladorJuego +refrescar(): void +getPanelNivel(): PanelNivel +gameOver(): void +victoria(): void +mostrarPanelVictoria(): void +mostrarPanelGameOver(): void +limpiarGraficaNivelActual(): void +mostrarPanelNombre(): void +getHelper(): HelperLecturaMovimiento

ConstantesGraficas
+ANCHO PANTALLA: int = 650 +ALTO PANTALLA: int = 640 +PANEL ANCHO: int = 650 +PANEL ALTO: int = 600

Sprite
#rutaSprites: String #estadoActual: EstadoSprite
+Sprite(ruta: String) +getRutalImagenActual(): String +getRutalImagen(estado: EstadoSprite): String +getEstadoActual(): EstadoSprite +setEstadoActual(estado: EstadoSprite): void

PanelVista
#controladorVista: ControladorVistas #fondo: Image
+PanelVista(controlador: ControladorVistas) #agregarImagenFondo(ruta: String): void #paintComponent(g: Graphics): void #transparentarBoton(boton: JButton): void #crearBotonRetro(texto: String, colorBase: Color): JButton

PanelNivel
#corazones: CopyOnWriteArrayList<JLabel> #labelPuntaje: JLabel #labelOleada: JLabel #labelTiempo: JLabel #labelNivel: JLabel
+PanelNivel(controlador: ControladorVistas) +agregarElemento(elemento: LogicElement): Observer +limpiarElementos(): void +agregarVida(): void +inicializarVidas(cantVidas: int): void +eliminarVida(): void -inicializarLabelTiempo(): void -inicializarLabelOleada(): void -inicializarLabelNivel(): void -inicializarLabelPuntaje(): void +actualizarTiempo(segundosTotales: int): void +actualizarOleada(oleada: int): void +actualizarNivel(numNivel: int): void +actualizarPuntaje(puntaje: int): void

PanelGameOver
#btnJugarDeNuevo: JButton #btnVerRanking: JButton #btnVolverMenu: JButton #lblPuntaje: JLabel
+PanelGameOver(cv: ControladorVistas) -inicializarComponentes(): void -crearBoton(texto: String): JButton +setPuntaje(puntaje: int): void +setListenerJugarDeNuevo(listener: ActionListener): void +setListenerVerRanking(listener: ActionListener): void +setListenerVolverMenu(listener: ActionListener): void

PanelMenu
#botonIniciar: JButton #botonPuntajes: JButton #botonClasico: JButton #botonPersonalizado: JButton #dominioSeleccionado: String = "CLASICO"
+PanelMenu(controlador: ControladorVistas) #agregarBotonIniciar(): void #agregarBotonPuntajes(): void #agregarBotonClasico(): void #agregarBotonPersonalizado(): void #agregarBotonPuntaje(): void #registrarOyenteBotonIniciar(): void #registrarOyenteBotonClasico(): void #registrarOyenteBotonPersonalizado(): void #registrarOyenteBotonPuntajes(): void -actualizarEstadoBotones(): void

PanelModos
#botonClasico: JButton #botonContrarreloj: JButton #botonSupervivencia: JButton #botonVolver: JButton
+PanelModos(controlador: ControladorVistas) #agregarBotonesModo(): void #agregarBotonVolver(): void #registrarOyenteBotonClasico(): void #registrarOyenteBotonContrarreloj(): void #registrarOyenteBotonSupervivencia(): void #registrarOyenteBotonVolver(): void #paintComponent(g: Graphics): void

PanelNombre
#campoNombre: JTextField #botonConfirmar: JButton #labelTitulo: JLabel #labelInstruccion: JLabel #sonido: ReproductorSonido = ReproductorSonido.getInstance() #nombreJugador: String
+PanelNombre(cv: ControladorVistas) -inicializarComponentes(): void -confirmarNombre(): void +limpiarCampo(): void

PanelRankings
#panelRankings: JPanel #botonVolver: JButton #rankingClasico: Ranking #rankingContraReloj: Ranking #rankingSupervivencia: Ranking
+PanelRankings(controlador: ControladorVistas) #agregarBotonVolver(): void #registrarOyenteBotonVolver(): void +agregarPanelRankings(): void -crearColumnnaRanking(titulo: String, ranking: Ranking, fuenteTitulo: Font, fuenteJugador: Font, colorTexto: Color, colorTitulo: Color): JPanel +getRankingClasico(): Ranking +setRanking(): void +actualizar(): void

PanelVictoria
#btnJugarDeNuevo: JButton #btnVerRanking: JButton #btnVolverMenu: JButton #lblPuntaje: JLabel
+PanelVictoria(cv: ControladorVistas) -inicializarComponentes(): void -crearBoton(texto: String): JButton +setPuntaje(puntaje: int): void +setListenerJugarDeNuevo(listener: ActionListener): void +setListenerVerRanking(listener: ActionListener): void +setListenerVolverMenu(listener: ActionListener): void

NoEstatico
#enSuelo: boolean #estado: Estado #direccionMirada: Vector2 #direccion: Vector2 = new Vector2(0, 0) #velocidad: Vector2 = new Vector2(0, 0)
+NoEstatico(posicion: Vector2) +setSprite(s: Sprite): void +enSuelo(): boolean +setEnSuelo(b: boolean): void +setEstado(e: Estado): void +getDireccion(): Vector2 +getDireccionMirada(): Vector2 +getVelocidad(): Vector2 +mover(): void

Proyectil
#GRAVEDAD: float = 0.005f #VELOCIDAD BASE: float = 7.0f #activa: boolean #aplicarGravedad: boolean
+Proyectil(posInicial: Vector2, direccion: Vector2) +Proyectil(posInicial: Vector2, direccion: Vector2, conGravedad: boolean) +estaActiva(): boolean +desactivar(): void +setSprite(s: Sprite): void +mover(): void +aceptarRegistro(r: Registrador): void +eliminar(): void +colisionar(P: PowerUp): void +colisionar(e: Enemigo): void +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +colisionar(e: Escalera): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void +colisionar(bolaf: BolaDeFuego): void +colisionar(bolan: BolaDeNieve): void +colisionar(bomba: Bomba): void +colisionar(fuego: Fuego): void

BolaDeFuego
+BolaDeFuego(posInicial: Vector2, direccion: Vector2) +aceptarColision(c: Colisionador): void

BolaDeNieve
#sonido: ReproductorSonido = ReproductorSonido.getInstancia() #doble: boolean
+BolaDeNieve(posInicial: Vector2, direccion: Vector2) +setSprite(s: Sprite): void +aceptarColision(c: Colisionador): void +dobleDaño(): boolean +colisionar(p: Plataforma): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +activarDoble(): void +eliminar(): void

Bomba
#direccion: Vector2 #estado: Estado
+Bomba(pos: Vector2, dire: Vector2) +mover(): void +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void +aceptarColision(c: Colisionador): void

Fuego
+Fuego(pos: Vector2, direccion: Vector2, largo: int) +aceptarColision(c: Colisionador): void +colisionar(bolaf: BolaDeFuego): void +colisionar(bomba: Bomba): void +colisionar(bolan: BolaDeNieve): void +colisionar(P: PowerUp): void +colisionar(e: Enemigo): void +colisionar(p: Plataforma): void +colisionar(sr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +colisionar(e: Escalera): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void +colisionar(fuego: Fuego): void

<i>Obstaculo</i>
+Obstaculo(posicion: Vector2)

Escalera
+Escalera(pos: Vector2) +aceptarColision(c: Colisionador): void

Pared
+Pared(pos: Vector2) +aceptarColision(c: Colisionador): void

ParedDestructible
#resistencia: int
+ParedDestructible(pos: Vector2) +RestarResistencia(): void +eliminar(): void +aceptarColision(c: Colisionador): void

SueloResbaladizo
+SueloResbaladizo(pos: Vector2) +aceptarColision(c: Colisionador): void

Trampa
+Trampa(pos: Vector2) +aceptarColision(c: Colisionador): void

<i>PowerUp</i>
#efecto: Efectos #activo: boolean #tiempoRestante: int <u>#tiempoDeVida: int = 500</u>
+PowerUp(posicion: Vector2) +decrementarTiempoDeVida(): void +getEfecto(): Efectos +eliminar(): void

PowerUpAzul
+PowerUpAzul(pos: Vector2) +aceptarColision(c: Colisionador): void

PowerUpDoblePuntaje
+PowerUpDoblePuntaje(pos: Vector2) +aceptarColision(c: Colisionador): void

PowerUpFruta
+PowerUpFruta(pos: Vector2) +aceptarColision(c: Colisionador): void

PowerUpRoja
+PowerUpRoja(pos: Vector2) +aceptarColision(c: Colisionador): void

PowerUpVerde
+PowerUpVerde(pos: Vector2) +aceptarColision(c: Colisionador): void

PowerUpVidaExtra
+PowerUpVidaExtra(pos: Vector2) +aceptarColision(c: Colisionador): void

Jefe
#puedeAtacar: boolean #puedeColisionar: boolean #saltando: boolean #cayendo: boolean #sufrioGolpe: int
+Jefe(posicion: Vector2) +restarResistencia(): void +sufrioGolpe(): boolean +puedeAtacar(): boolean +setPuedeColisionar(value: boolean): void +setSaltando(value: boolean): void +setCayendo(value: boolean): void +estaSaltando(): boolean +estaCayendo(): boolean +aceptarColision(c: Colisionador): void

Kamakichi
#cooldownAtaque: int #COOLDOWN_ATAQUE: int = 60 #CANON_SUPERIOR_IZQUIERDA: Vector2 = new Vector2(40, -20) #CANON_SUPERIOR_DERECHA: Vector2 = new Vector2(220, -20) #CANON_LATERAL_IZQUIERDA: Vector2 = new Vector2(-10, 50) #CANON_LATERAL_DERECHA: Vector2 = new Vector2(260, 50)
+Kamakichi(pos: Vector2) +mover(): void +lanzar(): void +accionarComportamiento(): void +setSprite(s: Sprite): void +colisionar(bolan: BolaDeNieve): void +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +tick(): void

Jugador
#lock: ReadWriteLock = new ReentrantReadWriteLock() #COOLDOWN: int = 10 #TIEMPO_PU: int = 300 #TIEMPO_INVENCIBLE: int = 120 #ESPERA_MUERTE: int = 60 #SONIDO_ESCALERA: int = 8 #vidas: int #tiempoInvencible: int #esperaMuerte: int #esperaSonidoEscalera: int #posicionInicial: Vector2 #puedeAtacar: boolean #powRoja: boolean #puntaje: int #enEscalera: boolean #registro: RegistroJugador #tiempoPU: int #velocidadActivada: boolean #sonido: ReproductorSonido = ReproductorSonido.getInstancia() #cooldownDisparo: int #doblePuntaje: boolean #reaparece: boolean #tipoAtaque: Ataque #animaciones: AnimationManagerJugador #sensorAtaque: SensorAtaqueJugador #detencionActivada: boolean #escalando: boolean #saltando: boolean #puedeSonarEscalera: boolean
+Jugador(posicion: Vector2) +getRegistro(): RegistroJugador +sumarPuntaje(p: int): void +getDoblePuntaje(): boolean +setSprite(s: Sprite): void +setJuego(j: Juego): void +getPuedeAtacar(): boolean +mover(): void +procesarInputMovimiento(dir: Vector2, saltoSolicitado: boolean): void +reaparece(): boolean +movio(): void +estaSaltando(): boolean +escalando(): boolean +enEscalera(): boolean +esInvencible(): boolean +atacar(): void #definirTipoAtaque(): void +getAtaque(): Ataque +lanzar(direccion: Vector2): void +patear(target: Enemigo, dir: float): void +perderVida(): void +reiniciarPosicion(): void +reiniciarEfectos(): void +detencionActivada(): boolean +aplicarPowerUp(e: Efectos): void #colisionConSuelo(suelo: GameElement): void +colisionar(p: PowerUp): void +colisionar(e: Enemigo): void +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +colisionar(e: Escalera): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void +colisionar(bolaf: BolaDeFuego): void +colisionar(bomba: Bomba): void +colisionar(bolan: BolaDeNieve): void +colisionar(fuego: Fuego): void -resolverColision(elemento: GameElement): void +getVidas(): int +aceptarRegistro(r: Registrador): void +eliminar(): void +getNombre(): String +setVidas(vidas: int): void +getPuntaje(): int +setPuntaje(puntaje: int): void +suscribe(suscriptor: Observer): void +notify(e: Eventos): void #timerTiempoInvencible(): void #timerAtaque(): void #timerEscalera(): void #timerPowerUp(): void #timerEsperaMuerte(): void #resetearInfoColision(): void +tick(): void

Moghera
#areaExcepcionColision: HitBox = new HitBox(0, 0, posicion.clone()) #hitboxGolpe: HitBox #puedeAtacar: boolean #muerto: boolean #cooldownAtaque: int #COOLDOWN_ATAQUE: int = 120
+Moghera(pos: Vector2) +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +lanzar(direccion: Vector2): void +setAreaSinColision(altura: float): void +accionarComportamiento(): void +setSprite(s: Sprite): void +colisionar(bolan: BolaDeNieve): void +morir(): void +estaMuerto(): boolean +tick(): void

GameElement
#observers: LinkedList<Observer> #WIDGHT: int #HEIGHT: int #hitbox: HitBox #sprite: Sprite #posicion: Vector2 #miJuego: Juego #puntos: int #zOrder: int
+GameElement(v: Vector2) +getWidth(): int +getHeight(): int +cambiarEstadoSprite(nuevoEstado: EstadoSprite): void +observers(): LinkedList<Observer> +suscribe(suscriptor: Observer): void +unsuscribe(suscriptor: Observer): void +notify(e: Eventos): void +setSprite(s: Sprite): void +getSprite(): Sprite +getPosicion(): Vector2 +getMijuego(): Juego +setJuego(j: Juego): void +getHitbox(): HitBox +getZOrder(): int +getPuntos(): int +eliminar(): void

HitBox
#base: float #altura: float #posicion: Vector2 #offset: Vector2
+HitBox(b: float, a: float, pos: Vector2) +HitBox(b: float, a: float, pos: Vector2, offset: Vector2) +getAltura(): float +getBase(): float +getPosition(): Vector2 +setPosition(v: Vector2): void +chequearColision(h: HitBox): boolean +calcularCorreccion(h: HitBox): Vector2 +diferenciaAltura(h: HitBox): int

Estatica
#tiempoActual: int
+Estatica(posicion: Vector2) +desplazar(): void +aceptarRegistro(r: Registrador): void +decrementarTiempoDeVida(): void +eliminar(): void

Decoracion
+Decoracion(posicion: Vector2)

Plataforma
+Plataforma(posicion: Vector2) +aceptarColision(c: Colisionador): void

PlataformaEstatica
+PlataformaEstatica(pos: Vector2) +aceptarColision(c: Colisionador): void

PlataformaMovil
#direccion: Vector2 #fueColisionada: boolean
+PlataformaMovil(pos: Vector2) +aceptarRegistro(r: Registrador): void +yaColisiono(): boolean +setFueColisionada(b: boolean): void +desplazar(): void +setDireccion(v: Vector2): void +getDireccion(): Vector2 +aceptarColision(c: Colisionador): void +colisionar(p: ParedDestructible): void +colisionar(p: Pared): void +colisionar(t: Trampa): void +colisionar(e: Escalera): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(p: Plataforma): void +colisionar(e: Enemigo): void +colisionar(p: PowerUp): void +colisionar(pm: PlataformaMovil): void +colisionar(bf: BolaDeFuego): void +colisionar(b: Bomba): void +colisionar(b: BolaDeNieve): void +colisionar(f: Fuego): void

PlataformaQuebradiza
#tiempoDeVida: int = 30 #fueColisionado: boolean
+PlataformaQuebradiza(pos: Vector2) +aceptarColision(c: Colisionador): void +decrementarTiempoDeVida(): void +pisoJugador(): void +eliminar(): void

Enemigo
#TIEMPO_DESCONGELAR: int = 50 #resistenciaMaxima: int #resistencia: int #estado: Estado #ia: IAEnemigo #detenido: boolean #animaciones: AnimationManager #tiempoDescongelar: int #puntosCongelado: int #enEscalera: boolean #eliminado: boolean #dirEmpuje: float #fueCongelado: boolean
+Enemigo(posicion: Vector2) +restarResistencia(): void +aceptarRegistro(r: Registrador): void +mover(): void +procesarSolicitudMovimiento(dir: Vector2): void +estaEnEscalera(): boolean +estaCongelado(): boolean +descongelar(): void +empujar(dir: float): void +eliminar(): void +resistenciaMaxima(): int +getResistencia(): int +getDireccionEmpuje(): float +setDetenido(d: boolean): void +estaDetenido(): boolean #aparecerPowerUp(): void #comporbarChoque(e: GameElement): void +colisionar(e: Enemigo): void +animar(): void +colisionar(P: PowerUp): void +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +colisionar(e: Escalera): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void +colisionar(bolaf: BolaDeFuego): void +colisionar(bolan: BolaDeNieve): void +colisionar(bomba: Bomba): void +colisionar(fuego: Fuego): void +getPuntos(): int #resolverColision(elemento: GameElement): void +tick(): void +accionarComportamiento(): void +aceptarColision(c: Colisionador): void

Animal
+Animal(posicion: Vector2)

DemonioRojo
+DemonioRojo(pos: Vector2)
+accionarComportamiento(): void
+setSprite(s: Sprite): void
+colisionar(e: Escalera): void

RanaDeFuego
#COOLDOWN_ATAQUE: int = 30
#sonido: ReproductorSonido = ReproductorSonido.getInstancia()
#cooldownAtaque: int
#puedeAtacar: boolean
+RanaDeFuego(pos: Vector2)
+lanzar(direccion: Vector2): void
+puedeAtacar(): boolean
+accionarComportamiento(): void
+setSprite(s: Sprite): void
+tick(): void

TrollAmarillo
#saltando: boolean
#cayendo: boolean
#puedeColisionar: boolean = true
#areaExcepcionColision: HitBox = new HitBox(0, 0, posicion.clone())
+TrollAmarillo(pos: Vector2)
+accionarComportamiento(): void
+setSprite(s: Sprite): void
+setSaltando(value: boolean): void
+setCayendo(value: boolean): void
+estaSaltando(): boolean
+estaCayendo(): boolean
+setAreaSinColision(altura: float): void
+setPuedeColisionar(value: boolean): void
+colisionar(p: Plataforma): void
+colisionar(pr: SueloResbaladizo): void
+colisionar(pq: PlataformaQuebradiza): void
+colisionar(pm: PlataformaMovil): void

Indestructible
+Indestructible(posicion: Vector2)
+colisionar(p: Proyectil): void

Calabaza
#puedeColisionar: boolean
+Calabaza(pos: Vector2)
+invocarFantasma(): void
+accionarComportamiento(): void
+animar(): void
+empujar(dir: float): void
+setPuedeColisionar(value: boolean): void
+aceptarColision(c: Colisionador): void
+colisionar(p: Proyectil): void
+colisionar(P: PowerUp): void
+colisionar(e: Enemigo): void
+colisionar(p: Plataforma): void
+colisionar(pr: SueloResbaladizo): void
+colisionar(e: Escalera): void
+colisionar(t: Trampa): void
+colisionar(p: Pared): void
+colisionar(p: ParedDestructible): void
+colisionar(bolaf: BolaDeFuego): void
+colisionar(bolan: BolaDeNieve): void
+colisionar(bomba: Bomba): void
+colisionar(fuego: Fuego): void
+colisionar(pq: PlataformaQuebradiza): void
+colisionar(pm: PlataformaMovil): void

Fantasma
+Fantasma(pos: Vector2)
+accionarComportamiento(): void
+animar(): void
+aceptarColision(c: Colisionador): void
+colisionar(P: PowerUp): void
+colisionar(e: Enemigo): void
+colisionar(p: Plataforma): void
+colisionar(pm: PlataformaMovil): void
+colisionar(pq: PlataformaQuebradiza): void
+colisionar(pr: SueloResbaladizo): void
+colisionar(e: Escalera): void
+colisionar(t: Trampa): void
+colisionar(p: Pared): void
+colisionar(p: ParedDestructible): void
+colisionar(bolaf: BolaDeFuego): void
+colisionar(bolan: BolaDeNieve): void
+colisionar(bomba: Bomba): void
+colisionar(fuego: Fuego): void

<i>FabricaSprites</i>
+obtenerSprite(tipo: TipoElementos): Sprite

FabricaGameElement
#fabricaSprites: FabricaSprites #registrador: ElementsRegister #miJuego: Juego
+FabricaGameElement(fs: FabricaSprites, j: Juego, registrador: ElementsRegister) +setFabricaSprites(fs: FabricaSprites): void +crearElemento(tipo: TipoElementos, posicion: Vector2): GameElement +crearProyectil(tipo: TipoElementos, pos: Vector2, dir: Vector2): Proyectil +getMiJuego(): Juego

FabricaClasica
#RUTA_SPRITES: String = "srcAssetsSprites//DominioClasico"
+FabricaClasica() +obtenerSprite(tipo: TipoElementos): Sprite

FabricaPersonalizada
#RUTA_SPRITES: String = "srcAssetsSprites//DominioPersonalizado"
+FabricaPersonalizada() +obtenerSprite(tipo: TipoElementos): Sprite

Efectos
NORMAL
VELOCIDAD
MATA_RAPIDO
DETENER_ENEMIGOS
MAS_PUNTAJE
VIDA_EXTRA
FRUTA
DOBLE_PUNTOS

EstadoSprite
BASE_IZQUIERDA,
BASE
BASE_DERECHA
SALTO_DERECHA
SALTO_IZQUIERDA
LANZAR_IZQUIERDA
LANZAR_DERECHA
EMPUJANDO_IZQUIERDA
EMPUJANDO_DERECHA
PATEAR_IZQUIERDA
PATEAR_DERECHA
MUERTE
ATURDIDO
IZQUIERDA
DERECHA
APARICION
ESCALANDO
EXPLOSION
CAYENDO_DERECHA
CAYENDO_IZQUIERDA
FINAL
DANIO
SUBIDA
ATAQUE

Eventos
ACTUALIZACION_GRAFICA
COLISION
ELIMINACION
PERDIDAVIDA
SUMA_VIDA
CAMBIO_RESISTENCIA
MUERTE
POW_AZUL
POW_ROJA
POW_VERDE
SUMA_PUNTOS
DOBLE_PUNTOS
SIN_EFECTOS
PUNTOS
EMPUJAR

Modo
CLASICO
CONTRARELOJ
SUPERVIVENCIA

TipoElementos
AIRE("00")
PLATAFORMA_ESTATICA("01")
PLATAFORMA_QUEBRADIZA("02")
PLATAFORMA_MOVIL("03")
SUELO_RESBALADIZO("04")
ESCALERA("05")
TRAMPA("06")
PARED("07")
PARED_DESTRUCTIBLE("08")
DEMONIO_ROJO("10")
TROLL_AMARILLO("11")
RANA_DE_FUEGO("12")
JUGADOR("20")
BOLA_DE_NIEVE("21")
POWER_UP_AZUL("22")
POWER_UP_VIDAEXTRA("23")
POWER_UP_ROJA("24")
POWER_UP_VERDE("25")
POWER_UP_FRUTA("26")
BOLA_DE_FUEGO("30")
CALABAZA("13")
FANTASMA("14")
POWER_UP_DOBLE_PUNTAJE("27")
ENEMIGO_ALEATORIO("15")
MOGHERA("16")
FUEGO("17")
KAMAKICHI("18")
BOMBA("19")

CreadorDeNiveles

#fabrica: FabricaGameElement
#rand: Random
#enemigosPosibles: List<TipoElementos>

+CreadorDeNiveles(fabrica: FabricaGameElement)
+cargarDesdeArchivo(rutaArchivo: String): Nivel
+cargarOleadaSupervivencia(nivel: Nivel, rutaArchivo: String): void
#cargarMetadatosNivel(nivel: Nivel, linea: String): void
#extraerDatos(linea: String): Map<String,Integer>
#cargarGameElementsEstaticos(nivel: Nivel, lineaElemento: String): void
#cargarGameElementsNoEstaticos(nivel: Nivel, lineaElemento: String): void
+obtenerLinea(archivo: String, linea: int): String
-cargarJugador(n: Nivel): void
+getJuego(): Juego

Launcher

+main(args: String[]): void

HiloJuego

#NS_POR_TICK: double = 1_000_000_000.0 / 30.0
#running: boolean = true
#modo: ModoDeJuego
#helper: HelperLecturaMovimiento
#timer: long
#ticks: int = 0

+HiloJuego(modo: ModoDeJuego, helper: HelperLecturaMovimiento)
+detener(): void
+run(): void
-tick(): void

AnimationManager
#raizAnimacion: NodoSelector #sprite: Sprite
+AnimationManager(sprite: Sprite) #armarArbolAnimacion(): void +animar(): void

AnimationManagerDR
#enemigo: DemonioRojo
+AnimationManagerDR(sprite: Sprite, enemigo: DemonioRojo) #armarArbolAnimacion(): void +animar(): void #seMueve(): boolean #animMover(): int #animQuieto(): int

AnimationManagerJugador
#jugador: Jugador
+AnimationManagerJugador(sprite: Sprite, jugador: Jugador) +animar(): void #armarArbolAnimacion(): void +estaEscalando(): boolean +reaparece(): boolean +estaMuerto(): boolean +estaSaltando(): boolean +estaAtacando(): boolean +ataqueLanzamiento(): boolean +ataquePatada(): boolean +estaCaminando(): boolean +animLanzar(): int +animPatear(): int +animSalto(): int +animCaminar(): int +animIdle(): int +animMuerte(): int +animReaparece(): int +animEscalar(): int

AnimationManagerKM
#enemigo: Kamakichi
+AnimationManagerKM(sprite: Sprite, enemigo: Kamakichi) #armarArbolAnimacion(): void +animar(): void #muerto(): boolean #atacar(): boolean #seMueve(): boolean #sufrioGolpe(): boolean #animMover(): int #animMuerto(): int #animAtacar(): int #animQuieto(): int #animGolpe(): int

AnimationManagerMO
#enemigo: Moghera
+AnimationManagerMO(sprite: Sprite, enemigo: Moghera) #armarArbolAnimacion(): void +animar(): void #muerto(): boolean #Final(): boolean #seMueve(): boolean #saltando(): boolean #cayendo(): boolean #sufrioGolpe(): boolean #animMover(): int #animSalto(): int #animCaer(): int #animMuerto(): int #animFinal(): int #animQuieto(): int #animGolpe(): int

AnimationManagerRF
#enemigo: RanaDeFuego
+AnimationManagerRF(sprite: Sprite, enemigo: RanaDeFuego) #armarArbolAnimacion(): void +animar(): void #seMueve(): boolean #atacando(): boolean #animMover(): int #animQuieto(): int #animAtaque(): int

AnimationManagerTA
#enemigo: TrollAmarillo
+AnimationManagerTA(sprite: Sprite, enemigo: TrollAmarillo) #armarArbolAnimacion(): void +animar(): void #seMueve(): boolean #saltando(): boolean #cayendo(): boolean #animMover(): int #animSalto(): int #animCaer(): int #animQuieto(): int

VisorJugadorJefes
#toleranciaVertical: float = 150
+VisorJugadorJefes(propietario: Enemigo, rango: float) +objetivoALaVista(): boolean

VisorJugador
#propietario: Enemigo #objetivo: Jugador #objetoEnMedio: boolean #rangoVision: float
+VisorJugador(propietario: Enemigo, rango: float) #actualizarPosicion(): void +objetivoEnRango(): boolean +objetivoALaVista(): boolean +obtenerAlturaJugador(): float +obtenerDireccionJugador(): int +obtenerPosicionJugador(): Vector2 +update(): void +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void

SensorManiobra
#detectaPisoArriba: boolean #detectaPisoAbajo: boolean #alturaPisoArriba: float #alturaPisoAbajo: float #margen: float
+SensorManiobra(propietario: GameElement, altura: float, margen: float) #actualizarPosicion(): void +update(): void +detectarPisoArriba(): boolean +detectarPisoAbajo(): boolean +getAlturaPisoArriba(): float +getAlturaPisoAbajo(): float +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void #calcularCentro(e: GameElement): Vector2

SensorEntorno
#jugador: Jugador #colisionaConPiso: boolean = false #colisionaConPared: boolean = false #escaleraParaBajar: boolean = false
+SensorEntorno(propietario: GameElement) +update(): void #actualizarPosicion(): void +detectaPiso(): boolean +detectaPared(): boolean +detectaEscaleraParaBajar(): boolean +getHitbox(): HitBox +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(e: Escalera): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void #calcularCentro(e: GameElement): Vector2

SensorAtaqueJugador
#enemigoCongelado: boolean #enemigoEnRango: Enemigo #jugador: Jugador
+SensorAtaqueJugador(propietario: Jugador) +update(): void #actualizarPosicion(): void +detectaEnemigoCongelado(): boolean +enemigoEnRango(): Enemigo +colisionar(e: Enemigo): void

Sensor
#propietario: GameElement #controladorColisiones: ControladorColisiones #hitbox: HitBox #posicion: Vector2 #centroPropietario: Vector2
+getHitbox(): HitBox +colisionar(bolaf: BolaDeFuego): void +colisionar(bomba: Bomba): void +colisionar(bolan: BolaDeNieve): void +colisionar(fuego: Fuego): void +colisionar(P: PowerUp): void +colisionar(p: Plataforma): void +colisionar(pr: SueloResbaladizo): void +colisionar(pq: PlataformaQuebradiza): void +colisionar(pm: PlataformaMovil): void +colisionar(e: Escalera): void +colisionar(t: Trampa): void +colisionar(p: Pared): void +colisionar(p: ParedDestructible): void +colisionar(e: Enemigo): void +update(): void

CapturadorGolpes
#posicionReferencia: Vector2
+CapturadorGolpes(p: GameElement, posReferencia: Vector2, width: float, height: float) #actualizarPosicion(): void +colisionar(bolan: BolaDeNieve): void +update(): void